AI Search on Databricks

embeddings · similarity · managed vs custom RAG · governance

① Embeddings

- An **embedding** = a list of numbers (a vector) that captures the meaning of text.
- Similar meaning → vectors **close together**, even with totally different words.

"cozy apartment near the park" ≈ "comfortable flat close to green space"

- When you create an index with `embedding_model_endpoint_name="databricks-gte-large-en"`, Databricks calls that model on **every chunk** to make its vector.

3 rules for embeddings ✎

- **Same model for documents AND queries** → otherwise they live in different vector spaces and can't be compared. (classic exam trap)
- **Respect the context window** (often ~512 tokens). Longer text is silently truncated.
- **Dimensions trade-off:** more dimensions = more nuance, but more storage + slower search.

② How similarity search works

Query text "pet-friendly?" → Embed [0.23, -0.41, ...] → Find nearest cosine similarity →

Top results
chunk 1 (0.94), chunk 2 (0.87)

1. Query is turned into a vector (same embedding model).
2. Index finds the **closest vectors** (cosine similarity or another distance metric).
3. Returns matching chunks **ranked by score**.

- This is what `similarity_search` does on an AI Search index.

- Databricks uses ANN (Approximate Nearest Neighbors): gives up a tiny bit of precision for a **big speed gain** at scale.

Cosine similarity in one picture

Angle between vectors	Score	Meaning
small (point the same way)	**close to 1.0** (e.g. 0.95)	very relevant
large (point different ways)	**toward 0** (e.g. 0.25)	not relevant

Cosine looks at **direction** (meaning), not length.

③ Managed RAG vs Custom RAG

Documents
PDFs, tables, UC Volumes
→
Managed: Knowledge Assistant
or
Custom: AI Functions + AI Search
→
Answers
LLM + retrieved context

	Managed RAG (Knowledge Assistant)	Custom RAG (Agent Framework + AI Search)
You give	docs in a UC Volume or an AI Search index	you build every step yourself
Who does parse / chunk / embed / index / serve	**Databricks, automatically**	**you**: ai_parse_document → ai_prep_search → Delta table → AI Search index
Control	low (but auto-improves via **feedback loops**)	full: parsing, chunking, embedding model, retrieval logic
How index is used	built in	AI Search index used as a **retrieval tool** in your agent
Best for	standard RAG, **fast to production**	unique needs, multi-step reasoning, new architectures

Both run the same pipeline underneath (parse → chunk → embed → index). Managed just does it for you.

④ Governance with Unity Catalog

UC Volume
source PDFs
→
Delta table
chunks + metadata
→
VS index
vectors + synced cols
→
VS endpoint
query interface
→
Agent
queries via endpoint

Piece	Controlled by
UC Volume, Delta table, **AI Search index**	Unity Catalog privileges
AI Search endpoint	endpoint ACLs (not UC privileges!)
Who did what	**audit** via system tables
Where data came from	**lineage** across UC tables + paths

One-line recap

- embed text → compare with cosine → ANN returns top chunks. Managed = Knowledge Assistant does it all; Custom = you build it. UC governs data + index, ACLs govern the endpoint.

Likely exam questions

- Retrieval returns junk after switching query embedding model? → **docs + queries must use the same model**
- Score 0.95 vs 0.25: which is more relevant? **0.95** (closer to 1)
- Why ANN instead of exact search? **speed at scale**, tiny precision loss
- Need a standard Q&A bot fast with least effort? **Knowledge Assistant (managed)**
- Need a custom chunking strategy / own embedding model? **Custom RAG**
- Restrict who can query the endpoint? **endpoint ACLs** · restrict the index / table? **UC privileges**

next lecture → send the screenshots